

Ch 5 Lecture 3

Intuition about SVD

Thinking again about matrix diagonalization

The same logic, applied to a general matrix

Recap

Many parts of this inspired by [this blog post](#)

$$A = USV^T$$

where

$$\begin{array}{ccc}
 U & & V \\
 \left(\begin{array}{c|c|c} | & & | \\ \mathbf{u}_1 & \cdots & \mathbf{u}_m \\ | & & | \end{array} \right) & \left(\begin{array}{c|c|c} | & & | \\ \mathbf{v}_1 & \cdots & \mathbf{v}_n \\ | & & | \end{array} \right) & \begin{array}{c} S \\ \left(\begin{array}{ccccccc} \sigma_1 & & & & & & \\ & \sigma_2 & & & & & \\ & & \ddots & & & & \\ & & & \sigma_r & & & \\ & & & & \ddots & & \\ & & & & & 0 & \end{array} \right) \end{array} \\
 \text{eigenvectors of } AA^T & & A^T A
 \end{array}$$

Example

$$A = \begin{pmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{pmatrix}$$

. . .

$$AA^T = \begin{pmatrix} 17 & 8 \\ 8 & 17 \end{pmatrix}, \quad A^T A = \begin{pmatrix} 13 & 12 & 2 \\ 12 & 13 & -2 \\ 2 & -2 & 8 \end{pmatrix}$$

$$AA^T = \begin{pmatrix} 17 & 8 \\ 8 & 17 \end{pmatrix}$$

eigenvalues: $\lambda_1 = 25, \lambda_2 = 9$
eigenvectors

$$u_1 = \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} \quad u_2 = \begin{pmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \end{pmatrix}$$

$$A^T A = \begin{pmatrix} 13 & 12 & 2 \\ 12 & 13 & -2 \\ 2 & -2 & 8 \end{pmatrix}$$

eigenvalues: $\lambda_1 = 25, \lambda_2 = 9, \lambda_3 = 0$
eigenvectors

$$v_1 = \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{pmatrix} \quad v_2 = \begin{pmatrix} 1/\sqrt{18} \\ -1/\sqrt{18} \\ 4/\sqrt{18} \end{pmatrix} \quad v_3 = \begin{pmatrix} 2/3 \\ -2/3 \\ -1/3 \end{pmatrix}$$

SVD decomposition of A :

$$A = U S V^T = \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{pmatrix} \begin{pmatrix} 5 & 0 & 0 \\ 0 & 3 & 0 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} & 0 \\ 1/\sqrt{18} & -1/\sqrt{18} & 4/\sqrt{18} \\ 2/3 & -2/3 & -1/3 \end{pmatrix}$$

Reformulating SVD

SVD as a sum of rank-one matrices

$$A = \sigma_l u_l v_l^T + \dots + \sigma_r u_r v_r^T$$

Back to our example

$$A = \begin{pmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{pmatrix}$$

...

Can decompose as

$$\begin{aligned}
 & \begin{array}{c} \sigma_i u_i v_i^T \\ \swarrow \quad \downarrow \quad \searrow \\ = 5 \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} & 0 \end{pmatrix} + 3 \begin{pmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \end{pmatrix} \begin{pmatrix} 1/\sqrt{18} & -1/\sqrt{18} & 4/\sqrt{18} \end{pmatrix} \\
 & \dots \\
 A &= \begin{pmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{pmatrix}
 \end{aligned}$$

Uses of SVD

Moore–Penrose pseudoinverse

Suppose we want to solve a linear system

$$Ax = b.$$

If A is invertible, the solution is

$$x = A^{-1}b.$$

...

But many matrices are **not invertible**:

- A may not be square,
- or it may have linearly dependent columns.

In this case, an exact solution may not exist. Instead, we look for the vector x that minimizes the **least-squares error**

$$\|Ax - b\|_2.$$

Among all least-squares solutions, we choose the one with smallest norm.

This motivates the **Moore–Penrose pseudoinverse** A^+ , defined so that

$$x = A^+b$$

is the minimum-norm least-squares solution.

Using the SVD

Take the singular value decomposition

$$A = U\Sigma V^T,$$

where

- U and V are orthogonal matrices,
- Σ is diagonal (possibly rectangular):

$$\Sigma = \begin{bmatrix} \sigma_1 & & & \\ & \ddots & & \\ & & \sigma_r & \\ & & & 0 \end{bmatrix}, \quad \sigma_1 \geq \cdots \geq \sigma_r > 0.$$

Normal equations

Least squares solutions satisfy the Normal equations

$$A^T A x = A^T b.$$

Substitute the SVD:

$$(V \Sigma^T U^T)(U \Sigma V^T)x = V \Sigma^T U^T b.$$

Since $U^T U = I$,

$$V \Sigma^T \Sigma V^T x = V \Sigma^T U^T b.$$

Multiply on the left by V^T :

$$\Sigma^T \Sigma V^T x = \Sigma^T U^T b.$$

Solve componentwise

$$\Sigma^T \Sigma V^T x = \Sigma^T U^T b.$$

We can write this componentwise as

$$\Sigma^T \Sigma \begin{bmatrix} \mathbf{v}_1^T \mathbf{x} \\ \mathbf{v}_2^T \mathbf{x} \\ \vdots \\ \mathbf{v}_n^T \mathbf{x} \end{bmatrix} = \Sigma^T \begin{bmatrix} \mathbf{u}_1^T \mathbf{b} \\ \mathbf{u}_2^T \mathbf{b} \\ \vdots \\ \mathbf{u}_n^T \mathbf{b} \end{bmatrix}.$$

Since both Σ^T and $\Sigma^T \Sigma$ are diagonal, they just multiply each element by the corresponding singular value (or square):

$$\Sigma^T \Sigma V^T \mathbf{x} = \Sigma^T U^T \mathbf{b}$$

becomes

$$\begin{bmatrix} \sigma_1^2 \mathbf{v}_1^T \mathbf{x} \\ \sigma_2^2 \mathbf{v}_2^T \mathbf{x} \\ \vdots \\ \sigma_n^2 \mathbf{v}_n^T \mathbf{x} \end{bmatrix} = \begin{bmatrix} \sigma_1 \mathbf{u}_1^T \mathbf{b} \\ \sigma_2 \mathbf{u}_2^T \mathbf{b} \\ \vdots \\ \sigma_n \mathbf{u}_n^T \mathbf{b} \end{bmatrix}.$$

In other words, we get a separate equation for each singular value σ_i :

$$\sigma_i^2 \mathbf{v}_i^T \mathbf{x} = \sigma_i (\mathbf{u}_i^T \mathbf{b}).$$

Case 1: $\sigma_i \neq 0$

$$\sigma_i^2 \mathbf{v}_i^T \mathbf{x} = \sigma_i (\mathbf{u}_i^T \mathbf{b}).$$

We can divide by σ_i^2 :

$$\mathbf{v}_i^T \mathbf{x} = \frac{1}{\sigma_i} \mathbf{u}_i^T \mathbf{b}.$$

Case 2: $\sigma_i = 0$

$$\sigma_i^2 \mathbf{v}_i^T \mathbf{x} = \sigma_i (\mathbf{u}_i^T \mathbf{b}).$$

The equation becomes

$$0 = 0,$$

which imposes no constraint on y_i .

To obtain the **minimum-norm solution**, we choose $\mathbf{x}$ to be orthogonal to $\mathbf{v}_i$, so that:

$$\mathbf{v}_i^T \mathbf{x} = 0.$$

Defining Σ^+

These two cases can be summarized by defining the **pseudoinverse of Σ**

$$\Sigma^+ = \begin{bmatrix} 1/\sigma_1 & & & \\ & \ddots & & \\ & & 1/\sigma_r & \\ & & & 0 \end{bmatrix}.$$

That's just replacing every non-zero singular value with its reciprocal.

Then the equation becomes

$$V^T \mathbf{x} = \Sigma^+ U^T \mathbf{b}.$$

The Moore–Penrose pseudoinverse

$$V^T \mathbf{x} = \Sigma^+ U^T \mathbf{b}.$$

Finally, we can multiply both sides on the left by V to get

$$\mathbf{x} = V \Sigma^+ U^T \mathbf{b}$$

The quantity $V \Sigma^+ U^T$ is acting like an inverse here – it is called the **Moore–Penrose pseudoinverse** of A .

For any vector $\mathbf{b}$,

$$\mathbf{x} = A^+ \mathbf{b}$$

is the minimum-norm solution to the least-squares problem.

Interpretation

$$\mathbf{x} = V \Sigma^+ U^T \mathbf{b}$$

The SVD reveals exactly what the pseudoinverse does:

- Each singular direction acts independently.
- Directions with $\sigma_i > 0$ can be inverted.
- Directions with $\sigma_i = 0$ contain information lost by A and cannot be recovered.

The pseudoinverse therefore

undoes the action of A wherever inversion is possible but ignores directions where information has been destroyed.

...

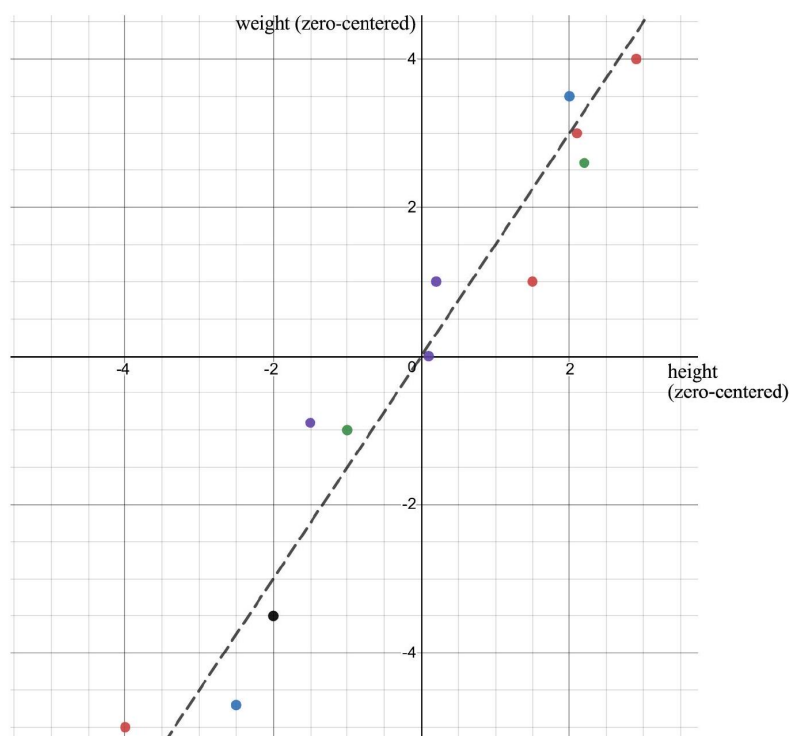
Example

Matrices as data

Example: Height and weight

$$A^T = \begin{bmatrix} 2.9 & -1.5 & 0.1 & -1.0 & 2.1 & -4.0 & -2.0 & 2.2 & 0.2 & 2.0 & 1.5 & -2.5 \\ 4.0 & -0.9 & 0.0 & -1.0 & 3.0 & -5.0 & -3.5 & 2.6 & 1.0 & 3.5 & 1.0 & -4.7 \end{bmatrix}$$

. . .



Why does $A^T A$ appear? (Data interpretation)

Up to now, matrices like $A^T A$ have appeared naturally during the construction of the SVD.

But suppose now that A is a **data matrix**.

What does $A^T A$ actually represent?

Step 1 — Interpret A as data

Think of

- rows = observations (people)
- columns = variables (height, weight, etc.)

Each row is one data point in $\mathbb{R}^p$.

Step 2 — Center the data

Subtract the mean of each column so every variable has mean 0.

We now assume:

A is zero-centered.

This assumption is the key bridge between linear algebra and statistics.

Step 3 — Examine one entry of $A^T A$

The (i, j) entry is

$$(A^T A) * ij = \sum *k = 1^n A_{ki} A_{kj}.$$

This computation:

- multiplies variable i and variable j together,
- then adds across all observations.

That is exactly how we measure **co-variation** between variables.

Step 4 — The covariance matrix appears

For zero-centered data,

$$\Sigma = \frac{1}{n-1} A^T A$$

is the **sample covariance matrix**.

So the matrix that appeared in the SVD derivation is not mysterious at all:

$A^T A$ measures how the variables vary together.

What do off-diagonal entries mean?

Look at a covariance matrix

$$\Sigma = \begin{bmatrix} \sigma_1^2 & c_{12} & \cdots c_{12} & \sigma_2^2 & \cdots \vdots & \vdots & \ddots \end{bmatrix}.$$

- The **diagonal entries** record how much each variable varies on its own.
- The **off-diagonal entries** measure how two variables move together.

Heights and weights example

Let's look at the (zero-centered) heights/weights dataset we've been using.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Data from "Example: Height and weight" ($A^T$): rows = height, weight; already zero-centered
heights = np.array([2.9, -1.5, 0.1, -1.0, 2.1, -4.0, -2.0, 2.2, 0.2, 2.0, 1.5, -2.5]) # inches
weights = np.array([4.0, -0.9, 0.0, -1.0, 3.0, -5.0, -3.5, 2.6, 1.0, 3.5, 1.0, -4.7]) # pounds

# Stack into an (n x 2) data matrix: columns = [height, weight]
X = np.column_stack([heights, weights])

# Center columns
Xc = X - X.mean(axis=0, keepdims=True)

# Sample covariance (divide by n-1)
Sigma = (Xc.T @ Xc) / (Xc.shape[0] - 1)

Sigma_df = pd.DataFrame(Sigma, index=["height", "weight"], columns=["height", "weight"])
Sigma_df.round(2)
```

	height	weight
height	4.86	6.67
weight	6.67	9.74

The off-diagonal entry $\Sigma_{12} = 6.67$ is the **covariance between height and weight**.

- The positive value of the off-diagonal entry means that (in this dataset) **when height increases, weight tends to increase too**.
- Knowing the height of an individual lets us make a guess about their weight.

But what if we rotated the axes?

A tilted data cloud suggests we are using a coordinate system that is not aligned with the natural shape of the data.

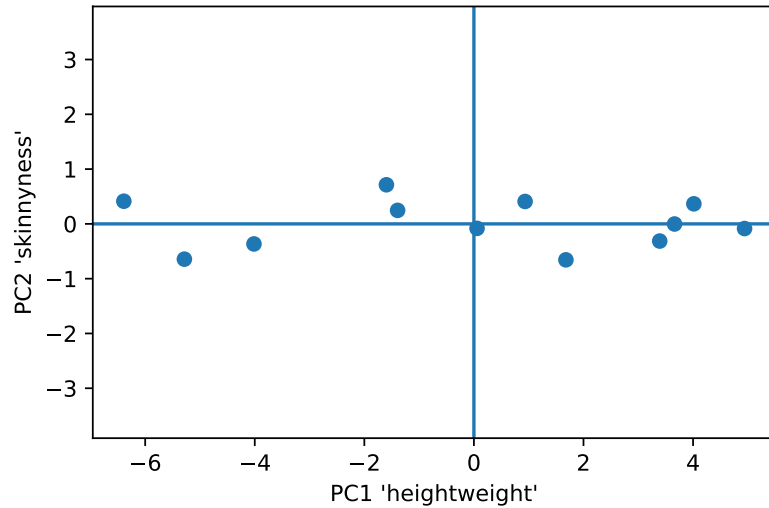
Let's rotate to new axes:

```
# Eigen-decomposition of the covariance matrix (Sigma is symmetric)
vals, vecs = np.linalg.eigh(Sigma)

# Sort eigenvalues descending (so first axis = biggest variance direction)
order = np.argsort(vals)[::-1]
vals = vals[order]
vecs = vecs[:, order]

# Rotate the centered data into the eigenvector basis
Y = Xc @ vecs

# Plot data in rotated (principal-axis) coordinates; axes are now PC1 and PC2
fig, ax = plt.subplots()
ax.scatter(Y[:, 0], Y[:, 1])
ax.set_xlabel("PC1 'heightweight'")
ax.set_ylabel("PC2 'skinnyness'")
ax.axhline(0)
ax.axvline(0)
ax.set_aspect('equal', adjustable='datalim')
plt.show()
```



In the rotated coordinate system, **moving along one new axis does not force movement along the other** (the axes line up with the ellipse).

The covariance matrix in the rotated coordinates

Now compute the covariance matrix of the rotated data Y .

```
Sigma_rot = (Y.T @ Y) / (Y.shape[0] - 1)
Sigma_rot_df = pd.DataFrame(Sigma_rot, index=["axis 1", "axis 2"], columns=["axis 1", "axis 2"])
Sigma_rot_df
```

	axis 1	axis 2
axis 1	1.440778e+01	9.285502e-16
axis 2	9.285502e-16	1.940396e-01

This covariance matrix is **diagonal** (up to tiny numerical rounding), which means:

- along these rotated axes, the two coordinates are uncorrelated,

- the variation in the data separates cleanly into two independent directions.

$$A^T = \begin{bmatrix} 2.9 & -1.5 & 0.1 & -1.0 & 2.1 & -4.0 & -2.0 & 2.2 & 0.2 & 2.0 & 1.5 & -2.5 \\ 4.0 & -0.9 & 0.0 & -1.0 & 3.0 & -5.0 & -3.5 & 2.6 & 1.0 & 3.5 & 1.0 & -4.7 \end{bmatrix}$$

...

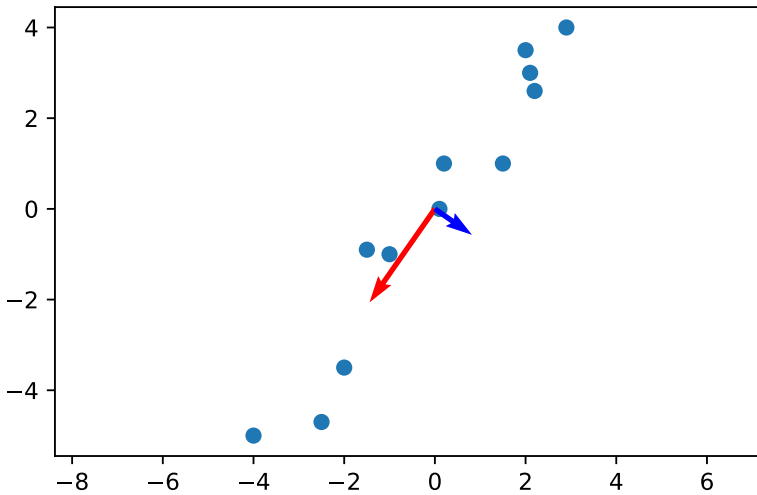
Plotting

The columns of the V matrix:

```
import numpy as np
import matplotlib.pyplot as plt
data = np.array([[2.9, 4.0], [-1.5, -0.9], [0.1, 0.0], [-1.0, -1.0], [2.1, 3.0], [-4.0, -5.0],
Ud,Sd,Vd=np.linalg.svd(data)
print(Vd)
plt.clf()

plt.scatter(data[:,0],data[:,1])
plt.quiver(0,0,Vd.T[0,0]*Sd[0]/5,Vd.T[1,0]*Sd[0]/5,angles='xy',scale_units='xy',scale=1, color
plt.quiver(0,0,Vd.T[0,1],Vd.T[1,1],angles='xy',scale_units='xy',scale=1, color = 'blue')
plt.axis('equal')
plt.show()
```

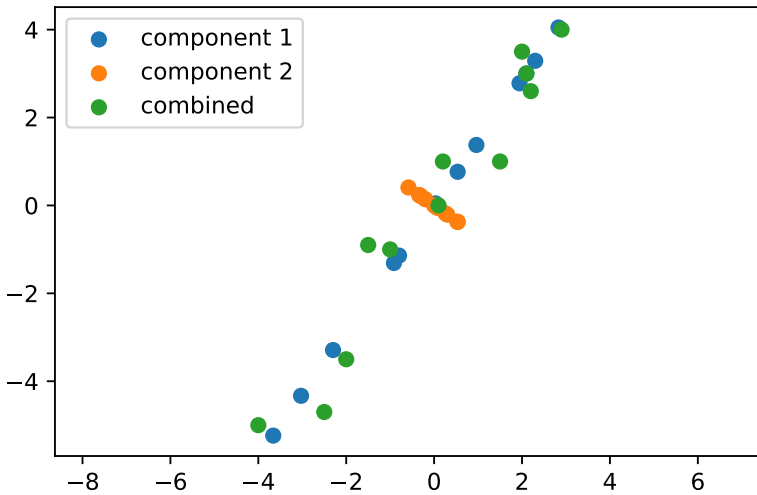
```
[[ -0.57294952 -0.81959066]
 [ 0.81959066 -0.57294952]]
```



Walk through why these vectors are actually the eigenvectors of the covariance matrix.

The components of the data matrix

```
#plt.scatter(data[:,0],data[:,1])
plt.clf()
component1 = np.outer(Ud[:,0],Sd[0]*Vd[0])
component2 = np.outer(Ud[:,1],Sd[1]*Vd[1])
plt.scatter(component1[:,0],component1[:,1])
plt.scatter(component2[:,0],component2[:,1])
combined = component1 + component2
plt.scatter(combined[:,0],combined[:,1])
plt.axis('equal')
# add a legend
plt.legend(["component 1", "component 2", "combined"])
plt.show()
```

The columns of the U matrix

```
plt.scatter(Ud.T[0]*Sd[0],Ud.T[1]*Sd[1])
plt.quiver(0,0,0,1,angles='xy',scale_units='xy',scale=1)
plt.quiver(0,0,1,0,angles='xy',scale_units='xy',scale=1)
plt.axis('equal')
# give x label "column u1" and y label "column u2"
plt.xlabel("column u1")
plt.ylabel("column u2")

plt.show()
```

